

NoHeadcountSystem

High-Performance Cryptocurrency HFT System

Your Name / Team Name

Your Institution / Department

2026 年 4 月 19 日



香港中文大學 (CUHK)
The Chinese University of Hong Kong, Shenzhen

- ① Team Division
- ② Project Background & Introduction
- ③ Core Architecture
- ④ Infrastructure
- ⑤ Strategies & Execution Layer
- ⑥ Risk Management
- ⑦ Backtest Engine
- ⑧ Dashboard & Deployment

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Team Division

表 1: Project Task Allocation

Team Member	Contribution
Mo Yunjie	Arbitrage Strategy, System Outline, Report
Li Taiyan	Infrastructure, OMS, GUI
Guo Xiaoyou	Event-Driven Backtesting
Huang Yujin	Risk Management, TCA

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Project Background & Introduction

- **System Positioning:** A High-Frequency Trading (HFT) system tailored for futures and cryptocurrencies.
- **Core Features:**
 - **Modular Design:** Decoupled infrastructure, strategies, risk management, and backtesting components.
 - **Event-Driven:** Non-blocking data and instruction routing based on a unified event bus.
 - **Seamless Switching:** One-click switch between simulated matching (SIM) and Binance Spot Testnet (BINANCE).
- **Tech Stack:** Python, Numba (High-performance computing), uv (Fast dependency management).

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Core Architecture & Data Flow

- **System Bus:** Modules are fully decoupled using the EventEngine.
- **Data Pipeline:**
 - ① **Tick:** Market data enters the system.
 - ② **Strategy:** Listens to Ticks and generates a Signal.
 - ③ **OMS (Order Management):** Converts Signals into standardized Orders.
 - ④ **RMS (Risk Management):** Intercepts and validates orders (limits, exposure).
 - ⑤ **Gateway:** Routes to the target exchange or simulator.
 - ⑥ **Trade:** Execution reports flow back via the event bus to update all modules.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Infrastructure Layer

- **Gateway Connection:**
 - `BinanceTestnetGateway`: Connects to Binance Spot Testnet for live testing.
 - `SimulatedGateway`: Local simulated matching engine for safe debugging.
- **Unified Data Models**: Standardized `TickData`, `Signal`, `OrderData`, `TradeData`.
- **Order Management System (OMS)**:
 - Bridges the strategy end (signals) and the channel end (orders).
 - Automatically tracks order status updates and maintains the signal-to-order mapping lifecycle.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer**
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Strategies & Execution Layer

- **Strategy Base (base.py):** Standardized interfaces where strategies only emit signals without placing orders directly, keeping responsibilities single and clean.
- **Featured Strategy: FeeAwareStatArbStrategy (dca.py)**
 - **Statistical Arbitrage:** Uses an Exponential Moving Average (EMA) to calculate micro-price deviations.
 - **Fee-Aware Logic:** Entries require the potential profit to strictly exceed double taker fees ($\text{taker_fee} \times 2$) plus a target profit rate.
 - **Dollar-Cost Averaging (DCA):** Dynamically adds to losing positions with strict step rates to lower the average cost.
 - **Smart Order Splitting:** Automatically splits position adjustments into legal CLOSE and OPEN offset signals.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Risk Management Center (RMS)

- **Real-time Interception System:** Acts as the last line of defense before orders are dispatched to the exchange (`manager.py`).
- **Core Risk Rules:**
 - **Max Order Volume** (`max_order_volume`): Prevents fat-finger errors and extreme anomalies.
 - **Max Net Position** (`max_symbol_position`): Controls unilateral exposure and inventory risks.
- **Dynamic Scalability:** The architecture allows for hot-plugging of new risk validation rules without altering the core pipeline.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

High-Performance Backtest Engine

- **Core Driver:** Numba-accelerated K-line backtesting core (exchange.py & loop.py).
- **Matching Mechanism:**
 - **Bar-Level Dual Sweep:** Simulates intra-bar volatility by scanning triggering limits through High and Low prices sequentially.
 - **Aggressive Limit Conversion:** Automatically converts marketable limit orders to market orders.
- **Accurate Accounting:**
 - Built-in Taker/Maker commission models.
 - Real-time maintenance of positions, cost prices, realized/unrealized PnL, and capital turnover.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Dashboard & GUI

- **Graphical User Interface:** Built via `order_monitor.py` to provide intuitive operational status.
- **Core Functions:**
 - **Real-time Metrics:** Overview of order volume, fills, and rejections.
 - **Dynamic Grids:** Tracks order lifecycles, trade reports, and tick streaming.
 - **Event Timeline:** Visual log display of system events for easy debugging.
- *Supports headless mode (`ENABLE_GUI=0`) for cloud server deployment.*

Quick Start

- **Minimal Dependencies:** Uses `uv` (Rust-based Python package manager) for instant environment syncing (`uv sync`).
- **Config-Driven (`.env`):**
 - `TRADING_MODE=SIM/BINANCE` for one-click environment switches.
 - Centralized configuration for API keys, proxies, and strategy parameters.
- **One-Command Run:** `uv run src/main.py` automatically initializes all modules and starts the event loop.

- 1 Team Division
- 2 Project Background & Introduction
- 3 Core Architecture
- 4 Infrastructure
- 5 Strategies & Execution Layer
- 6 Risk Management
- 7 Backtest Engine
- 8 Dashboard & Deployment

Conclusion & Future Work

- **System Advantages:**
 - Clear isolation of responsibilities (signals decoupled from orders).
 - High extensibility for new exchanges and strategies.
 - Consistent logic between backtesting and live trading.
- **Future Extensions:**
 - Integrate more exchange gateways (e.g., OKX, Bybit).
 - Introduce Machine Learning models for alpha signal prediction.
 - Enhanced backtest charting and detailed attribution reports.

Q&A / Thank You!